Pseudoknot Finder

Predict secondary structures that include pseudoknots, the crossing base pairs that standard MFE folding cannot represent, using the bundled Knotty engine and its DP09 energy model.

Overview

Standard RNA folding (ViennaRNA, and the main RSAS pipeline) assumes nested base pairs only. Every pair sits cleanly inside or outside every other pair, which is what makes the usual dot-bracket notation work: a `(` always matches the next unclaimed `)`, with no crossing.

A pseudoknot breaks that assumption. It happens when a base pair opens inside one stem but closes outside it, so two stems cross each other:

```
Nested:      (((...)))           pairs do NOT cross
Pseudoknot:  (((...[[...)))...]]  the () and [] CROSS
```

In the pseudoknot above, the `[` opens inside the `()` stem but the `]` closes outside it. Crossing pairs cannot be written with a single pair of brackets, so Knotty emits extended dot-bracket notation that uses additional bracket families (`[]`, `{}`, `<>`) to mark each crossing level. This is the structural feature that the rest of RSAS deliberately does not model, which is why the Pseudoknot Finder is a separate tool.

The Pseudoknot Finder predicts these structures with Knotty, which uses the DP09 energy model from HotKnots V2.0. RSAS calls the binary once per sequence and reports, for each one, whether a pseudoknot is present, the predicted structure, and the minimum free energy.

When to use it

Reach for the Pseudoknot Finder when nested folding is not enough:

- You are studying riboswitches, structured regulatory RNAs, or other elements where crossing pairs are part of the mechanism.
- The main pipeline's folded structure looks incomplete and you suspect a pseudoknot is being missed (the main pipeline cannot represent one even if it exists).
- You want a yes/no answer on whether a set of candidate sequences contains pseudoknots, ranked by free energy.

Keep the cost in mind. Pseudoknot prediction is far more expensive than nested folding, and the cost grows steeply with sequence length (see How it works). This tool is built for short to medium sequences, not whole transcripts. By default anything over 500 nt is skipped.

Step by step

1. Open the **Pseudoknot Finder** page (titled "Pseudoknot Finder, Knotty").
2. Check the status banner at the top. Green means the Knotty binary was found and is ready. Red means it was not found, with the expected path in the message (see Troubleshooting).
3. Choose your input:
   - If you already loaded sequences in the main app, they carry over automatically. The Input section shows a count like "N sequences loaded from main app".
   - To use a file instead, click **Browse** and pick a FASTA file, or paste a path into the **FASTA file** field. A FASTA path overrides the loaded sequences (see Options in detail).
4. Set the parameters if the defaults do not suit you:
   - **Timeout (sec)**, default `120`.
   - **Max length (nt)**, default `500`.
5. Click **Run Pseudoknot Prediction**. Progress and results stream into the log area below. While a run is in progress, **Cancel** becomes active.
6. When the run finishes, read the results table in the log (Name, Length, Energy, PK?, Structure). Pseudoknotted sequences are listed first, then sorted by energy.
7. Click **Export CSV** to save the full results to a file.

If you need to stop early, click **Cancel**. The batch stops after the sequence it is currently folding finishes, so you keep partial results. Closing the dialog mid-run prompts for confirmation and signals the same cancel.

Options in detail

Timeout (sec)

- **Default:** 120 seconds, **per sequence** (not per batch).
- **Effect:** If a single sequence does not finish folding within this many seconds, that sequence is reported as an error (the message notes it timed out and may be too long) and the batch moves on to the next one. A slow sequence cannot stall the whole run.
- Raise it if you are folding longer sequences and seeing timeouts you want to push through. Lower it if you would rather fail fast and not wait on stragglers.
- If you type something that is not a whole number, the field falls back to 120.

Max length (nt)

- **Default:** 500 nt.
- **Effect:** Sequences **longer** than this are **skipped entirely**, not folded. They never reach the Knotty engine. This is the most important option to understand, because skipped sequences quietly drop

out of the results. The log reports how many were skipped (for example, "Skipped 3 sequences exceeding 500 nt"), but each skipped sequence produces no row of its own.
- The cap exists because pseudoknot prediction cost grows with the fourth power of length. Raising it can turn a quick run into one that takes minutes per sequence, so raise it only when you are prepared to wait.
- If every loaded sequence exceeds the cap, the run does not start. You get an "All Skipped" warning telling you all N sequences exceed the max length.
- If you type something that is not a whole number, the field falls back to 500.

Note there is also a hard wrapper-level ceiling of 2000 nt enforced beneath the GUI. Even if you set Max length above 2000, any individual sequence past that ceiling is rejected by the wrapper with a "Sequence too long" error rather than folded.

Sequence source priority

The finder draws sequences from one of two sources, and the order is fixed:

1. **FASTA file (override).** If the FASTA field has a path, that file is parsed and used, and the loaded sequences are ignored. The field placeholder says as much: "optional, overrides loaded sequences".
2. **Loaded sequences.** If the FASTA field is empty, the sequences carried over from the main app are used.

If both are empty, the run does not start and you get a "No Sequences" warning. The FASTA parser reads standard `>` headers (taking the first whitespace-delimited token as the name) and concatenates the sequence lines; it warns if the file is larger than 100 MB.

What you get

For each sequence that was folded, the finder reports:

- **Pseudoknot present?** A yes/no flag. It is set whenever the predicted structure contains any crossing-level bracket (`[` , `]` , `{` , `}` , `<` , or `>`). A purely nested structure (`(` and `)` only) reads as no.
- **Predicted structure.** Extended dot-bracket notation:
  - `( )` standard nested base pairs.
  - `[ ]` pairs that cross with `( )` pairs (first crossing level).
  - `{ }` pairs that cross with `( )` or `[ ]` pairs (second level).
  - `< >` a further crossing level.
- **Minimum free energy (MFE)**, in kcal/mol, from the DP09 model.

In the on-screen results table the columns are **Name**, **Len** (length), **Energy**, **PK?**, and **Structure** (truncated for display). Rows are sorted pseudoknots-first, then by energy ascending. Sequences that

errored (for example, timed out) are listed separately under an Errors heading with their error message.

CSV export columns

Export CSV writes one row per result with these columns:

Column	Contents
`Name`	Sequence name
`Length`	Length of the cleaned sequence
`Sequence`	The cleaned RNA sequence actually folded
`Structure`	Full predicted structure in extended dot-bracket
`Energy_kcal`	MFE in kcal/mol (blank if the sequence errored)
`Has_Pseudoknot`	`Yes` or `No`
`Error`	Error message if the sequence failed, otherwise blank

The CSV includes error rows too, so failures are visible in the file rather than silently missing. Cell values that begin with a spreadsheet formula character are prefixed with a single quote to prevent formula injection when the file is opened in Excel.

How it works

The page is a thin GUI over a subprocess wrapper around the Knotty command-line binary.

- `check_knotty_available()` (in `RnaThermofinder/utils/knotty_wrapper.py`) locates the binary, checking the frozen app bundle first, then the project's `bin/<platform>/` directory, then the system PATH, and makes it executable. The status banner reflects its result.
- When you click Run, the GUI filters out sequences longer than Max length, then calls `run_knotty_batch()`, which runs Knotty sequence by sequence on a background thread so the interface stays responsive. A `cancel_event` lets Cancel stop the loop between sequences.
- For each sequence, the wrapper uppercases the input, converts any `T` to `U` (Knotty expects RNA, ACGU), strips IUPAC ambiguity codes (with a warning), and rejects empty or over-2000-nt input. It then invokes the binary with the `-w` flag for minimal output.
- `_parse_knotty_output()` extracts the structure and energy from Knotty's stdout and sets the pseudoknot flag by scanning the structure for any crossing-level bracket. The parsed fields land in a `KnottyResult` (`seq_name`, `sequence`, `structure`, `energy`, `has_pseudoknot`, plus raw output and error fields).

Knotty implements the DP09 energy model (from HotKnots V2.0) and predicts complex pseudoknotted structures with a dynamic-programming algorithm whose cost grows steeply with sequence length,

from O(n^4) up to about O(n^6) for more complex pseudoknot classes. That scaling is the reason for the Max length skip and the per-sequence timeout: at the O(n^4) floor, doubling the length multiplies the work roughly sixteenfold, and more for harder pseudoknots.

For the engine's place in the wider analysis, and the version pinned for reproducibility, see ../methods.md.

Worked example

Suppose you carried three candidate leader sequences over from the Analyze page and want to know which fold into pseudoknots.

1. Open the Pseudoknot Finder. The banner is green and the Input section reads "3 sequences loaded from main app".
2. Leave Timeout at 120 and Max length at 500. All three sequences are under 500 nt, so none will be skipped.
3. Click **Run Pseudoknot Prediction**. The log shows "Starting pseudoknot prediction on 3 sequences..." and then per-sequence progress.
4. The results table appears, pseudoknots first:

   ```
   Name                  Len   Energy  PK?  Structure
   -------------------- ----- -------- ----  ------------------------------
   candidate_2            58   -21.40  YES  (((((...[[[[...)))))......]]]]
   candidate_1            42   -15.20   no  (((((....((((....))))....)))))
   candidate_3            61    -9.80   no  ..(((......)))...((....))......
   ```

   `candidate_2` folds into a pseudoknot: the `[[[[` opens inside the `(((((` stem and `]]]]` closes after it, so the two stems cross. The other two are nested.

5. Click **Export CSV**, choose a location, and you get a file with the full structures, energies, and a `Has_Pseudoknot` column you can filter on.

The energies above are illustrative. Your own runs will produce the actual DP09 values for your sequences.

Tips

- **Triage with a tight Max length first.** Run with the default 500 nt cap to clear the short sequences quickly, then raise the cap for the few long ones you actually care about, accepting the longer wait.
- **Use Timeout as a circuit breaker.** If you are unsure how long sequences will take, a moderate timeout lets the batch report the slow ones as errors instead of blocking, and you can re-run just

those with a higher timeout.

- **Read the skip count in the log.** Before trusting that "everything folded", check the "Skipped N sequences" line. A missing sequence is almost always a length skip, not a failure.
- **Carry sequences over from Analyze** rather than re-pointing at a FASTA, unless you specifically want a different set. Remember the FASTA field overrides loaded sequences whenever it is non-empty.
- **Pseudoknot pairs are predicted less accurately than nested pairs**, so treat a predicted pseudoknot as a hypothesis to confirm, not a settled structure.

Limitations and gotchas

- **The 500 nt skip is silent per sequence.** This is the big one. Sequences over Max length never get folded and produce no result row. Only an aggregate "Skipped N" line in the log tells you they were dropped. If long sequences seem to vanish from your results, the Max length cap is why. Raise it (and wait) if you truly need them.
- **Cost grows steeply with length (O(n^4) up to O(n^6)).** A sequence twice as long takes at least roughly sixteen times as long to fold, and more for complex pseudoknots. This is the reason the cap and timeout exist, and the reason this tool is not meant for whole transcripts.
- **macOS-only bundled binary.** The app ships a `knotty` binary for macOS. On Linux (and Windows through WSL2) build Knotty from source and copy the `knotty` executable and its `simfold/` folder into `bin/linux/` of your RSAS folder; no administrator rights are needed (INSTALL.md, section 8). This is the same platform caveat as RNArobo.
- **RNA-only energies.** The DP09 model folds RNA in isolation. Ligand-RNA interactions, which matter for riboswitches, are not modeled, so a riboswitch's ligand-bound conformation is not what you are seeing.
- **Pseudoknot accuracy is lower than nested accuracy.** Sensitivity for pseudoknotted pairs is meaningfully below that for nested pairs. Predicted crossing pairs deserve more skepticism than predicted nested pairs.
- **Inputs are cleaned before folding.** `T` becomes `U`, and IUPAC ambiguity codes (`R`, `Y`, `S`, `W`, `K`, `M`, `B`, `D`, `H`, `V`, `N`) are stripped with a warning. The `Sequence` column in the export shows the cleaned sequence that was actually folded, which may be shorter than your raw input.
- **A hard 2000 nt ceiling sits below the GUI.** Even with Max length set higher, any single sequence past 2000 nt is rejected by the wrapper rather than folded.

Troubleshooting

The banner is red, "Knotty not found". The binary could not be located in the app bundle, the project's `bin/<platform>/` directory, or on PATH. The message includes the expected path. On macOS the binary ships with the app, so a red banner there usually means a broken or relocated install. On other platforms, build Knotty from source (`https://github.com/HosnaJabbari/Knotty`) and

copy `knotty` and `simfold/` into `bin/<platform>/` of your RSAS folder (INSTALL.md, section 8). Clicking Run while the banner is red shows a "Knotty Not Found" error and does nothing else.

Everything got skipped ("All Skipped" warning, or no result rows). Every loaded sequence is longer than Max length. Either raise Max length (and accept the longer run time) or supply shorter sequences. Check the skip count in the log to confirm this is a length issue and not an input problem. If you provided a FASTA, also confirm it parsed: a malformed file can yield zero sequences, which produces a "No Sequences" warning instead.

Sequences are timing out. A timed-out sequence appears under the Errors heading with a message noting it may be too long. Raise the Timeout to give Knotty more time, or lower Max length so the expensive long sequences are skipped before they ever run. Because cost scales with the fourth power of length, a sequence only a little longer than the rest can take far longer, so isolating and re-running just the slow ones with a higher timeout is often the most efficient fix.

No sequences to run ("No Sequences" warning). Neither a FASTA path nor loaded sequences were available. Load sequences in the main app first, or point the FASTA field at a file.

Reference

Jabbari, H. et al. (2018). Knotty: efficient and accurate prediction of complex RNA pseudoknotted secondary structures. *Bioinformatics* 34(22):3849-3856.